快讯：GPT-6 发布了


当你被导师压力说上周什么都没做
你突然想起了一个
不属于你的力量
Ultra
GPT-6 Astra
Select effort

但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ✦ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
  - ✦ 实现会反过来改变需求；设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ✦ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill Structured Problems)
- ✦ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束；今天还要骂，并不意味着这会是永久的能力边界。

“理应当做到”的事就是能做到

我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理

- 直接杀死了比赛 😂 (具身赛道的人应该在瑟瑟发抖)



做了小半年的工作直接🈚了

Baseline 应该是不可能比得过了。心疼 PhD Student 一分钟😭😭😭 #我爱工作工作爱我 #一想到上班就崩不住

地点 | 泰康之家养老社区 · 苏园 · 1.8km

Anyway... 我们还是要学习的

创造力需要正确的概念和理解

对世界**建立正确的概念和理解**，是具有发散和创造能力的前提。

> ✦ 专家的知识围绕核心概念组织；这样的组织方式帮助我们在新情境中运用知识。How People Learn (2000)

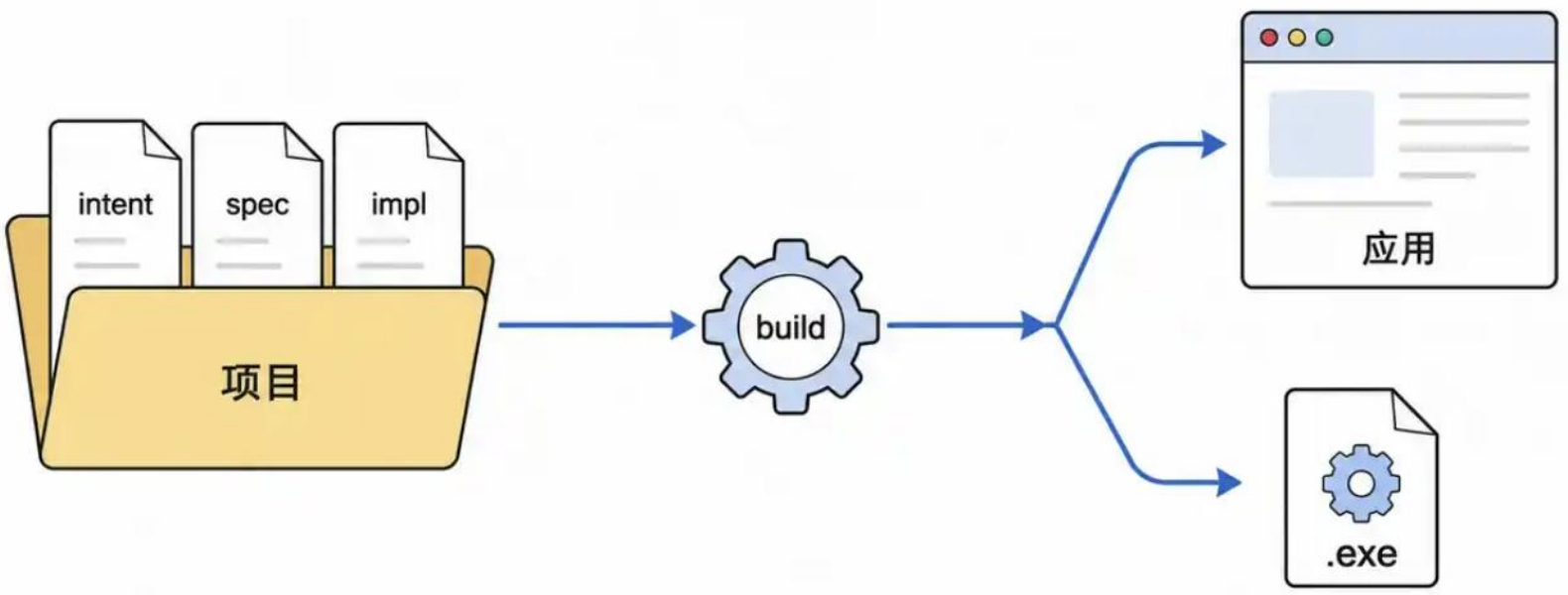
今天：进入真正的软件工程任务

从零开始构建一个项目。首先，什么是一个**软件工程项目**?

什么是一个软件工程项目？

一个目录，一组文件，一个可交付的东西

- 目录里的文件，混合了 intent、spec、impl、……
- 经过 build / 运行，可以得到 `npm run dev` 的本地窗口，也可以是一个 `.exe`。



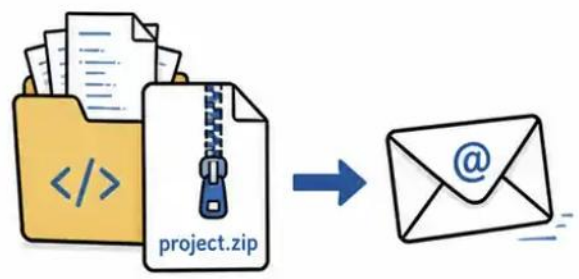
古早时期：That's All

一个优盘，装着全部代码

- 我的第一个 PhD Student 的故事（约 2012）：大一把代码保存在优盘上；优盘坏了，代码没了。
- 交作业：打包发邮件；现在似乎变成了网页上传。



优盘坏了，代码没了



打包 → 发邮件

今天：Git

项目版本管理有了标准答案

✦ 保存当前文件之外，还能保留、比较和恢复历史版本。Pro Git

作业发布与提交也很容易

- `git clone`：获取作业项目。
- `git pull`：获取后续更新。
- `make submit`：提交作业。

✦ `make submit` 是课程项目约定的提交入口，由项目里的 Makefile 定义。GNU Make

学习变容易以后，学习意识更重要

大量别人的项目，就是你的学习素材

- 公众号上满天都是 GitHub 链接。
- 哪怕没有人告诉你哪个好、哪个不好，看得多了，也会有感觉。

“上课耽误学习”的本质

课外的信息量、信息密度巨大，但我们从未教过**如何用好**。

✦ 把一次浏览变成一个具体问题：这个项目里的哪一个细节，是我自己做时不会想到的?

留意：让每一次阅读都成为学习

Virtually Implement：先在脑中做一遍

- 看到问题，先问自己：我会怎么想？如何 approach?
- 再看作者的做法：
- 如果一致——很好。
- 如果不一致——就是一个学习的机会。

对细节保持注意力

- 读别人的东西，甚至 AI 生成的东西，都要有“感知细节”的意识。
- 哪怕只看到一段话，也可以预测下一段是什么。
- **留意**使你持续学习、水平持续上升。

✦ 对照时再追问一句：“这个差异来自什么约束或假设？”学习研究中的“自我解释”，也关注把解题步骤与背后的原理联系起来。Chi et al. (1989)

Git：听说过，还是用过?

上周的保研面试

- 解释什么是 Git：多多少少都能说两句。
- `git rebase` 是什么：0/10。
- `git merge` 冲突以后，会观察到什么?
  - 竟然没有人说出：文本内容冲突时，双方内容和冲突标记会直接写入工作区文件；`git status` 显示 `both modified`。

第一没用过，第二没有留意

如果平时刷各种材料，多多少少会刷到；真正用过，也要留意眼前发生的事。

✦ 观察入口：打开冲突文件，看看 Git 留下了什么。

Git 之前，版本怎么管?

先把旧的留下来

- ✦ 最直接：复制整个目录，叫 作业-v1、作业-v2、作业-最终版。
  - ✦ 能找回旧文件；但“哪份从哪份改来、谁改了什么”，要靠人记。
- ✦ 用工具记：RCS 等工具替你保存文件的历史版本。

再让大家一起改

- ✦ CVS、SVN：把历史放在一台服务器，大家取出文件、修改，再交回去。
- ✦ Git：保存项目的一个个版本，也记录它们的来路，帮助大家汇合改动；
- 通常每个人本地就有完整历史。

✦ 版本管理的几种做法 · Pro Git

Git 从哪里来?

一个大项目，逼出了自己的工具

Linux 内核：改动怎样汇到一起?



需求一直在：保存来路，让许多人的改动汇成一个项目。

Git 简史 · Pro Git

我们要的是项目级“回溯时间”

人和 Agent 都会犯错误

- 设计会被改掉，对的代码会被改错.....
- 哪怕手滑把文件删掉了，只要已保存在 Git 历史中，就能捞回来。

Git 管理的是项目的快照

- ✦ 一次提交，记下纳入版本管理的文件当时的内容，以及它们在目录里的位置。
- ✦ 为什么要以项目为单位？代码和配置一起改过，只恢复旧代码、留下新配置，可能拼出一个从未正常运行过的版本。
- ✦ “存过盘”不等于“存进历史”：希望将来回到的状态，需要提交保存。

✦ 快照与提交 · Pro Git

从已有经验，长出 Git 的概念

“现在的你”是一个知识网络

学习的方式，是自然地“外推”它，这样可以帮你记忆。

- ✦ 你知道改错了想撤销 → 项目也需要能回去的旧状态。
- ✦ 你知道几个文件要配套 → 应当记住项目在同一时刻的样子。
- ✦ 你知道多人会同时修改 → 需要知道各自从哪里出发，才能汇合改动。

让“快照”成为推出来的需求

✦ 试着预测：删除一个已经提交过的文件，再提交一次，旧版本里还能找到它吗？能——新快照里没有它，旧快照仍然记着它。

从空白开始

Tutorial 告诉你的基本模型

`git init` 会得到一个空的 repository (repo)。可以使用一些不知道什么意思的命令，和 GitHub、GitLab、……建立关联。

- `git clone`：从远处下载代码
- `git add`：可以“添加”到 Staging Area (Index)
- `git commit`：可以“提交”一个快照

就可以开工了?

✦ `git add hello.c` 之后，又改了 `hello.c`，下一次 commit 会提交哪个版本?

✦ 提交的是 **add 时暂存的内容**。会运行一个命令，和知道它改变了什么，是两回事。Git 官方说明

获取下一级知识

学会追问，学习才进入正轨

绝大部分人都倒在了“获取下一级知识”的路上。

- ✦ “能运行”以后：为什么别人还要写 `.gitignore`?
- ✦ “文件都在”以后：为什么换一台电脑就编译不了?

Best Practice 从哪里来?

软件和系统都有**约定俗成**的 best practice，而每一个都是血的教训，或者对旧设计忍无可忍的产物。

- ✦ 把源码和编译产物一起提交：源码改了，产物忘了更新——同一件事有了两份互相矛盾的状态。
- ✦ 把自己的绝对路径写进配置：在我电脑上能跑，到了同学那里就失效。

✦ **“目录里有什么”和“项目需要共同保存什么”，是两个问题。**

多问一句：为什么、该不该?

哪怕只是放一个 hello.c

缺少足够的知识，也会犯错误。这时候，对细节的“注意力”就非常重要了。

> AGENTS.md: 在做任何 git 操作之前，都逐个文件检查变化，并为我 (新手) 解释是否符合 best practice，为什么，以及相关的概念。

让解释长出下一次判断的能力

- ✦ hello.c：程序的源码，要保存。
- ✦ hello：如果是编译出来的可执行文件，通常不提交；保存源码和构建方法，让别人也能生成它。
- ✦ 接着追问：这个文件是项目的输入，还是构建、运行留下的产物？别人拿到仓库，是否具备重建它的条件?

进组以后的高血压时刻

我和许多研究生新生都说过

- 生成的文件不提交 (`dist/`、log、……)
- 个人的配置不提交 (`.vscode` 中的个人设置等)
- ✦ 判断的是用途：团队共享、可移植的配置可以提交，不能只看目录名。
- 密钥不能提交 (审稿的时候见过好多)
- 临时文件不提交 (`.DS_Store`、……)
- 规则是活的： package lock, generated header, ...

忽略不等于遗忘

- ✦ `.gitignore` 不会让已经被跟踪的文件自动退出版本管理。
- ✦ 密钥一旦泄露，删掉当前文件也不能收回别人拿到的副本；应先撤销或轮换凭据，再处理历史。GitHub 指南

大写还是小写?

真实项目中的一个选择

```
[TYPO-CHECK] Docs should be typo-free
v.s.
[typo-check] Docs should be typo-free
```

Check-rule 的名字应该是大写还是小写?

直觉也值得追问

我的直觉是“大写似乎更好，但不是极强”，但不太说得出原因。

✦ 先想一想：方括号里的字符，是一句话的一部分，还是一个可以被引用的规则名字?

这里，我会选 TYPO-CHECK

用字形提示不同的角色

- ✦ [TYPO-CHECK] 是固定的规则标识；Docs should be typo-free 是解释规则的自然语言。大写让两者在视觉上更容易分开。
- ✦ 讨论“违反了 TYPO-CHECK”时，这个短名字也更像一个明确的引用。这是此处的设计判断，并非“大写总是更可读”。

约定赋予字形含义

- ✦ 一个类比：RFC 8174 明确，MUST 等关键词仅在大写时承载约定的特殊含义，避免与普通行文混淆。
- ✦ 它没有规定 check-rule 必须大写。项目若已有小写规则体系，或工具要求小写，应优先保持一致。

✦ 所以“大写似乎更好，但不是极强”：这里有让规则标识更醒目的理由，也有比这点偏好更重要的上下文。

告别古法学习

计算机科学定律：你想要的就一定有人做

- 把关键细节以直观的方式呈现出来
- 人类的视觉系统是非常高效的
    - 软件可视化已经几十年进展缓慢，然后，忽然所有问题都解决了

把真实的 Git Repo 展开给你看

> 实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动；明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。

直接生成，现场对比

- GPT-6 直出 v.s. https://learngitbranching.js.org/; 国模直出 v.s. https://code.cs61a.org/

从看懂到学懂

- ✦ 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。Hundhausen 等，2002

愿意学懂，成了真正有价值的品质

“把细节做好” 的 Bar

- 大学里没训练过，真上了战场就都得死
- 很多大学老师真正的好东西打开都没打开过，就只会照本宣科、断章取义
    - 我真的无法接受看公众号学习依然心安理得的老登

好消息：真正学懂和成绩是正相关的

- **愿意学懂**，就成了真正有价值的品质。

遗憾的是，自己的学习时间非常有限



更多的 Best Practices

比如，你会写 Commit Message 吗?

- Conventional Commits
- ✦ `fix(parser): accept repeated spaces between arguments`
  - ✦ 类型 + 范围 + 行为变化：人可以浏览，工具可以分类、生成 changelog。

Diff 留下改动，Message 留下理由

- ✦ 正文示例：旧解析器把连续空格当成空参数，导致复制来的命令失败；现在忽略多余空格，保留引号内空格。
- ✦ `fix`：只能说明“修了 bug”；触发条件、设计理由和 issue 链接，才能帮助后来的人判断“这段代码能不能删”。

Best Practices：让错误可恢复

一、给自己留退路

还有随处可见的 best practice。

- ✦ **及时 commit**：实验前留检查点；只在工作区里的修改，Git 未必能救。
- ✦ **备份与恢复**：误操作后先保护现场，再查 reflog；远端 clone 不包含未提交文件。

二、把探索整理成别人能接手的变化

- ✦ **发布前整理**：按一个意图拆分提交，把修复和相应测试放在一起；顺手格式化另开提交。
- ✦ **让历史可检验**：整理后的提交能构建、能测试，review、revert、bisect 才有可靠的单位。

Best Practices：让协作可预期

三、稳定共同依赖的东西

- ✦ **不擅自改写别人依赖的历史**：改写会把整理成本转嫁给协作者；修错通常追加提交。
- ✦ **约定分支与发布流程**：谁合并、如何回移补丁、何时打 tag；共同规则让交接可预期。

四、让约定有执行机制

- ✦ **Hooks + CI**：本地尽早反馈，服务端守住合入条件；同步上游后重新测试组合结果。
- ✦ **连接 issue、commit 和审查**：从“为什么改”能找到“改了什么”，再找到“如何验收”。

✦ 我的分类：降低丢失、理解和协调成本。原文也强调，具体 workflow 取决于项目。Git Best Practices

SWE-Bench：接手已有项目的一次修改

输入是项目现场，输出是补丁

然后你会发现，这些任务和软件工程日常任务是关联非常大的。

- ✦ 经典 SWE-bench 从真实 GitHub issue 和对应 PR 构建任务：给定仓库、起始提交 `base_commit` 和问题描述，产出 patch。任务定义
- ✦ 日常工作顺序：理解现象 → 复现 → 找调用路径和已有约定 → 修改 → 验证 → 检查 diff。

修好这个问题，同时守住已有行为

- ✦ `FAIL_TO_PASS`：针对问题的测试，由失败变为通过。
- ✦ `PASS_TO_PASS`：原来通过的测试，修改后仍然通过。评测规则
- ✦ 小而完整的改动便于定位失败；固定起点让别人能重放修改。前面的习惯，在这里都有用。

一行修复，背后是一整套工程工作

真实任务：Astropy 的嵌套模型

- ✦ SWE-bench 任务 `astropy__astropy-12907`：把两个独立线性模型先打包再组合，依赖矩阵却把它们算成互相依赖。Issue #12906
- ✦ 根因：拼接矩阵时，把右侧子模型的依赖矩阵填成了全 1；修复为保留原矩阵。代码改动

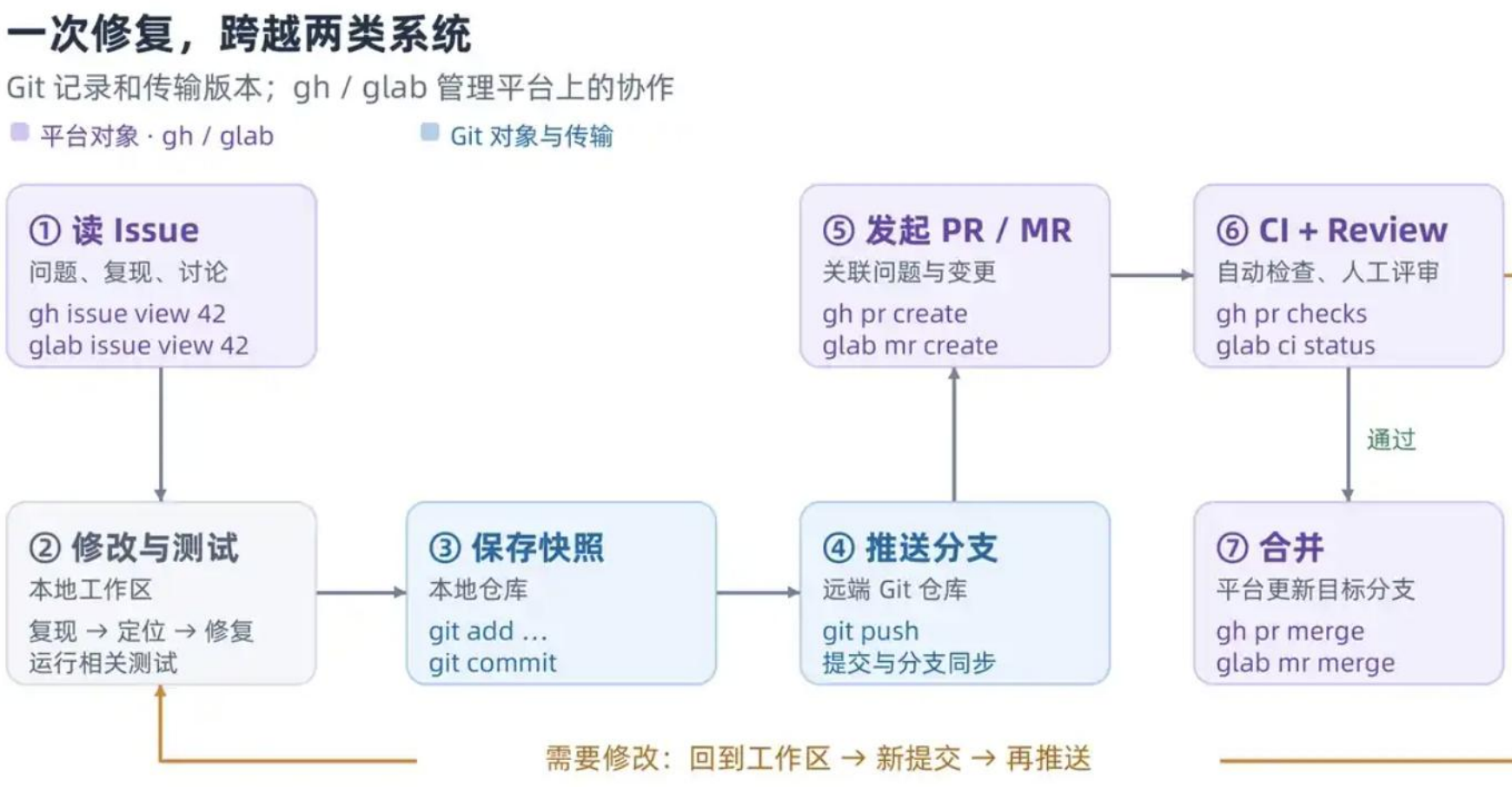
“改对”之后，怎样让项目接受?

- ✦ 真实 PR 还补了不同嵌套方式的测试、changelog；经过 review 合并，再回移到 `v5.0.x`。PR #12907
- ✦ 代码解释怎么修，测试留下证据，changelog 告知用户，回移把修复送到仍在维护的版本。
- ✦ SWE-bench 主要验收补丁的测试结果；现实项目还要完成沟通、审查和交付。

Git 之外：gh 与 glab

从问题到合入，把流程接起来

甚至为了管理 non-git 的部分，还专门有 gh, glab 这些工具。



git push 传输的是版本；PR / MR 还承载讨论、检查与合并决策。

✦ Git 保存和传输版本；gh / glab 操作平台上的 issue、PR/MR 与 CI 状态，让人和 Agent 都能从命令行完成协作。

回到软件工程

项目 = 由 Atomic Change 推进的快照

可以分支，可以合并。这是人类成功的经验。

- 人无法处理“一团混沌”
- 必须分解成可以理解、可以审计的小任务
    - 出了问题可以 bisect，发生冲突可以解决
    - 有些项目要求 linear history：历史易于追溯，cherry-pick / blame 友好

AI-native 也要避免“一团混沌”

- 在无限大能力模型到来之前，这条法则依然成立
- ✦ **一个 change 对应一个意图**：出了问题，才能单独解释、验证、撤销。

Agent 能力之内的项目

架构设计：单线程，反复讨论

比如 Pico Playground：和高智力模型讨论，直到收敛。

- 将设计落到文档、规则、示例：比如测试要做到什么程度
- 仍然要学操作系统、编译原理、数据库：这里有人类积累的好设计
- ✦ 架构先确定接口与不变量，后续任务才有可以独立行动的边界。

实现：有限并发，立即反馈

- $n \leq 3$ 个 terminal
- AGENTS.md 明确协作规则，例如 linear history
- 人脑并发控制：布置不冲突的任务
- 不管 AI 的质量如何，都立即检查、视觉验收、反馈

当 Agent 快了 1000 倍

一个思想实验：代码产能暴涨

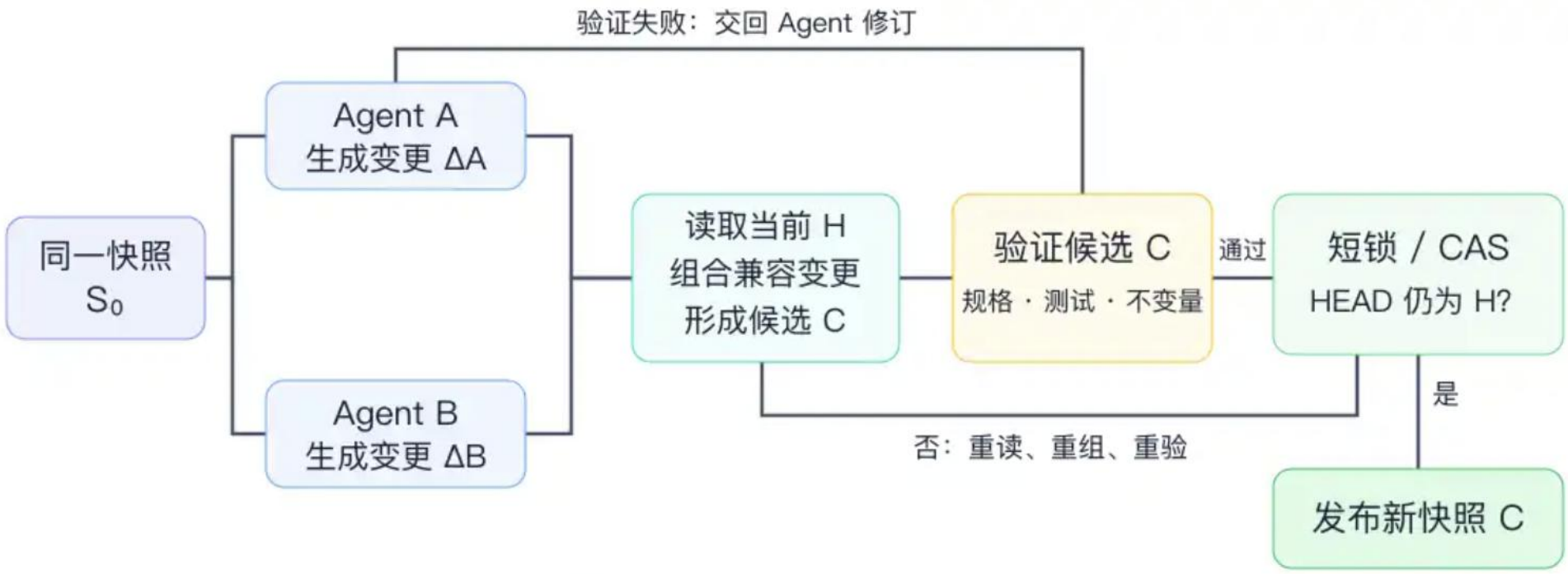
- Agent 可以 1000X 更快地生产代码
- Git“为人设计”的使用方式，可能跟不上这个速度
  - ✦ 生成一分钟，人工验收一小时：积压的是等待判断的变更。
  - ✦ 即使修改不同文件，也可能同时改变同一个接口约定。

从零设计一个 Agent VCS

- 快照、变更，以及一个 change algebra
- ✦ 系统要回答：哪些变更能组合、哪些必须排队、哪些需要重新验证?
- ✦ 像数据库处理事务一样，让 Agent 提交“基于某个版本的修改”，由系统控制它何时生效。

快照、变更与并发控制

在快照上并行工作，在发布时决定顺序



MVCC 管理并发版本；Change Algebra 描述变更如何组合

✦ **MVCC** 管理多个版本；**change algebra** 描述变更怎样组合。可交换的变更满足 $\Delta_B(\Delta_A(S)) = \Delta_A(\Delta_B(S))$，但组合结果仍需满足项目的不变量。

✦ 图为一种设计设想：生成与测试在锁外进行；只对验证过的确切版本做原子发布。版本已变，就重新组合、验证。参考：PostgreSQL 并发控制。

合并成功之后，还要问什么？

Jujutsu 已经靠近这个方向

- ✦ 稳定的 change ID：一个变更经过修改、rebase，仍能追踪其身份。见 Jujutsu Tutorial。
- ✦ 冲突可以作为状态保存；operation log 记录仓库操作，并支持并发操作的协调。见 First-Class Conflicts 与 Operation Log。

没有文本冲突，也可能违反约定

- ✦ 项目要求：至少一名值班员在岗。快照中，甲、乙都在岗。
  - ✦ Agent A 看到乙在岗，把甲改成离岗；Agent B 看到甲在岗，把乙改成离岗。
  - ✦ 即使修改分别落在两个文件里，合并后也会无人值班。
- ✦ 数据库里，这叫 **write skew**：只提供快照还不够，需要约束验证或更强的并发控制。参考：事务隔离。

Agent 时代：项目管理会变成什么？

一种可能性：大步历史，极致的正确性保障

- Agent 通过 lock / unlock 获得快照，完成修改、验证后进入
  ✦ 主线；锁只保护取版本和发布，工作在独立快照中进行。
- ✦ 人定义意图、不变量、验收标准；Agent 生成候选变更，系统依据验证结果准入。
- ✦ 人只看有意义的大步历史；机器保留细粒度操作和验证证据，供定位、重放、回退。

“出什么问题修好就行”

- 甚至“审计”可能也不是问题了：反正没有 Agent 能被开除、能坐牢
  ✦ 这个设想成立的关键：**错误能否在造成不可逆后果前被发现，修复是否足够便宜?**
- ✦ 测试全绿只是已有验收条件通过。验证越强、恢复越便宜，越能放手让 Agent 大步前进。
- ✦ 审计的重心可能转向：**系统凭什么接受这个变更？缺失的约束是什么?**